

Obsah

Poděkování	1
1 Předmluva ke druhému vydání	7
2 Úvodní informace	8
2.1 Ochranné známky	8
2.2 Vyloučení odpovědnosti	8
2.3 Licence ukázkových příkladů	9
2.4 Typografie	9
2.5 Různé	9
3 Úvod a motivace	10
3.1 Proč vznikla tato kniha?	10
3.2 Co budeme potřebovat?	11
3.3 Proč se učit programovat?	11
3.4 Proč se učit právě jazyk Java?	12
3.5 Proč webové aplikace?	13
3.6 Pár slov o jazyku Java	13
4 Základní pojmy	15
4.1 Data	15
4.2 Algoritmus	15
4.3 Paměť	18
4.3.1 Ukládání čísel	18
4.3.2 Ukládání znaků	19
4.3.3 Ukládání logických hodnot	19
4.3.4 Jak počítač rozumí informacím v paměti	19
4.4 Proměnná	20
4.5 Primitivní datové typy a přetypování	20
4.6 Java výrazy a příkazy	22
4.7 Stromová struktura dat	24
4.8 Model reálného světa	25
4.9 Fyzikální objekt	25
4.10 Programový objekt	25
4.11 Třída objektů	26
4.11.1 Diagram modelu tříd	26
4.11.2 Název třídy	27
4.11.3 Atributy	28
4.11.4 Metody	28

4.11.5 Konstruktor	29
4.11.6 Třída ve zdrojovém kódu	29
5 Od třídy k reálnému kódu	31
5.1 Užití objektu	31
5.1.1 Texty v jazyce Java	31
5.1.2 Vytvoření instance a užití třídy	31
5.1.3 Autoboxing	33
5.1.4 Neplatné objekty null	34
5.1.5 Vztahy mezi třídami	35
5.2 Další poznámky ke třídám	38
5.2.1 Viditelnost metod, atributů a tříd	38
5.2.2 Dědičnost	39
5.2.3 Výjimky	42
5.2.4 Balíčky	45
5.2.5 Komentáře kódu a JavaDoc	46
5.2.6 Interface	48
5.2.7 Generické datové typy	49
5.2.8 Inline třídy a lambda výrazy	50
5.2.9 Třída typu Record	51
5.2.10 Knihovny	52
5.3 Datový typ pole	52
5.3.1 Pole vytvořené metodou třídy	53
5.4 Vybrané třídy standardní knihovny	54
5.4.1 Třída String	54
5.4.2 Rozhraní List	55
Třída ArrayList	56
List s typem prvku	57
5.4.3 Rozhraní Map	57
5.4.4 Rozhraní Stream	58
5.4.5 Třída BigDecimal	59
5.4.6 Shrnutí kapitoly o třídách	61
6 Webové technologie a pojmy	62
6.1 XML – rozšiřitelný značkovací jazyk	63
6.2 HTML – hypertextový značkovací jazyk	65
6.3 CSS – Kaskádové styly	66
6.4 URL – Adresa internetové stránky	69
6.5 JSON – úsporný datový formát	70

6.6 Shrnutí	71
7 Řešené příklady	72
7.1 Instalace	73
7.1.1 Hardware a operační systém	73
7.1.2 Apache Maven	74
7.1.3 Znakový terminál	75
7.1.4 Získání příkladů	75
7.1.5 Instalace vývojového prostředí	75
7.1.6 Spouštění příkladů	76
7.1.7 Co dělat při potížích?	76
7.1.8 Virtuální počítač	77
7.1.9 Virtualizační kontejner	78
7.1.10 Vývojové editory	79
7.2 Servlety	79
7.2.1 Hello, World!	80
První servlet	81
Výsledek servletu	84
7.2.2 Datový model HTML stránky	84
Připojení knihovny do projektu	87
7.2.3 Programové výrazy	88
7.3 Tvorba tabulek	90
7.3.1 Jednoduchá tabulka	90
Příkaz for	91
Inkrementace a dekrementace	92
Řešení úkolu	92
Alternativní procházení pole	94
Podmíněný příkaz if-else	94
Tabulka náhodných čísel	95
7.3.2 Přehled vozidel	97
7.4 Formuláře	99
7.4.1 Formulář s jedním vstupním elementem	99
7.4.2 Formulář s více vstupními elementy	101
Regulární výrazy	102
Formulář osobních údajů	103
Ternární operátor	106
Hodnocení platnosti textů	107
7.4.3 Počítání slov	108

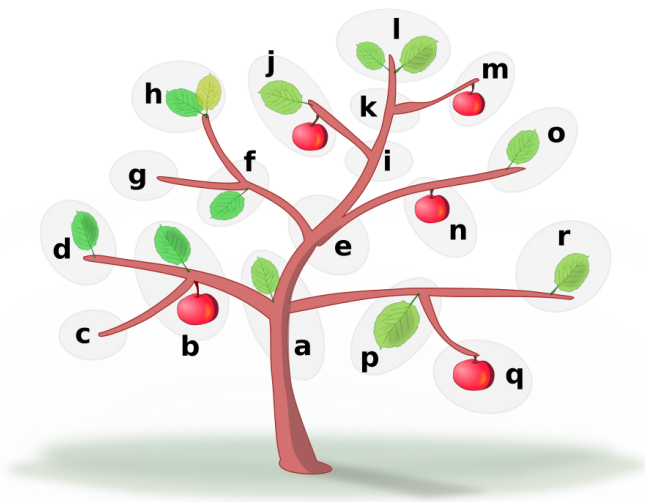
7.5 Zábava	110
7.5.1 Vlastní rodič servletu	110
7.5.2 Bodové kreslení	111
Pomocná třída Base64Converter	113
Parsování textu	113
Class – třída pro popis tříd	114
Logování událostí	114
Třída Optional	115
7.5.3 Binární podoba textu	115
Řešení	116
7.5.4 Piškvorky s defenzivní strategií hry	118
Diagram tříd	119
Jak to funguje?	119
Enumerátor	121
Příkaz switch	123
Popis tříd diagramu a jejich metod	124
Zpracování dotazu	125
7.6 Interaktivní aplikace s AJAX	126
7.6.1 Co je to AJAX	127
7.6.2 Testování regulárních výrazů	128
7.6.3 Piškvorky s rychlejší odezvou	131
7.7 Herní strategie od umělé inteligence	134
7.7.1 Jak formulovat požadavek na herní strategii pro AI	134
7.7.2 Závěrečné kroky a autorská práva	135
7.7.3 Automatizované testy	135
7.8 Verzování zdrojového kódu	136
8 Úvod do databází a práce s daty	139
8.1 Základní pojmy relačních databází	139
8.2 Relace a entity	140
8.3 Jazyk SQL aneb jak mluvit s databází	141
8.3.1 Tvorba databázové tabulky	141
8.3.2 Vložení řádku	142
8.3.3 Čtení řádku	143
8.3.4 Změna záznamu	144
8.3.5 Mazání řádku	144
8.3.6 Databázové transakce	145
8.4 Programové rozhraní Javy	145

8.4.1 Základní rozhraní JDBC	145
8.4.2 Pokročilé rozhraní JPA	146
8.4.3 Jak usnadnit práci s JDBC	147
8.5 Hledání hotelů v jazyce Java	147
8.6 Diagram tříd	148
8.7 Životní cyklus databázového spojení	149
8.8 Čtení databáze v Java objektech	151
8.9 Vkládání Java objektů do databáze	153
8.10 Generická abstraktní třída	154
9 Jak psát dobrý kód	156
Reference a zdroje pro další studium	158
Rejstřík slov	161

Výsledkem porovnání je logická hodnota typu `boolean`. Zde jsou uvedeny jen vybrané operátory. Vlastní preference lze vynutit kulatými závorkami. Primitivní typy jsou výhodné pro svou nízkou paměťovou náročnost a přímou podporu operátorů. Další příklady najdete v kapitole [Programové výrazy](#).

4.7 Stromová struktura dat

Data lze seskupovat do struktur, které zrychlují vyhledávání nebo usnadňují manipulaci. Se **stromovou** strukturou se budeme setkávat často, proto si představíme základní pojmy.



Obrázek 5. Schéma stromu

Pro lepší představu o tomto datovém modelu si vezměme na pomoc strom jabloně s jejími listy a plody. Místo, kde se větve dělí, se nazývá *uzel* (anglicky *node*). Tento pojem obecně označuje jakékoliv místo, ke kterému se připojují další potomci. Na obrázku jabloně to jsou další větve, plody či listy stromu.

Koncový uzel, který již nemá žádné potomky, se nazývá *list* (anglicky *leaf*). Každý uzel má právě jednoho *rodiče*, s výjimkou prvního uzlu, kterému říkáme *kořen* (anglicky *root*). Uzlům i listům můžeme přiřadit názvy. Spojnice mezi dvěma uzly se nazývá *hrana*. Přesnou adresu uzlu zapíšeme jako postupný seznam názvů od kořene, například oddělený tečkou.

```
a.e.f.h.žlutý_list
```

Podobně bychom mohli lokalizovat i červené jablko vpravo nahoře.

a.e.i.k.m.jablko

Stromovou strukturu má také adresa na poštovní obálce (stát → město → ulice). Jiným příkladem může být *hierarchická klasifikace organismů*, dalších příkladů by se našlo určitě více.

4.8 Model reálného světa

Vysvětlení pojmu *model* uvedeme krátkou definicí:

Model je zjednodušená reprezentace určitého objektu reálného světa či systému – pojatá z určitého úhlu pohledu.

— inspirováno zdrojem [10]

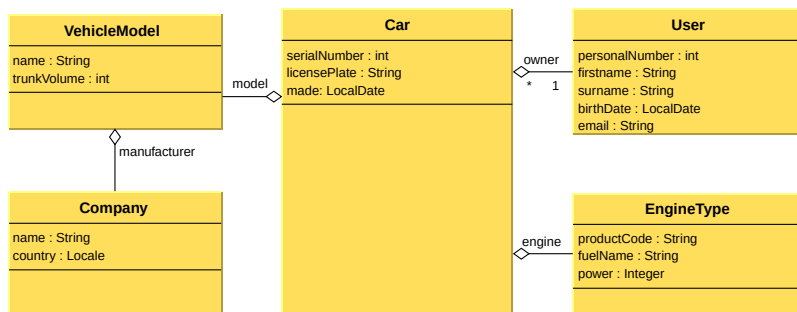
Děti si hrají s modely autíček nebo s panenkami, což jsou v napodobeniny předmětů reálného světa. Často se zachovává tvar či barva, zatímco jiné vlastnosti jsou potlačeny kvůli bezpečnosti nebo ceně. Pokud se model skládá z informací pro počítačové zpracování, říkáme mu *datový model*.

4.9 Fyzikální objekt

Při programování si *objekt* můžeme představit jako digitální model *tělesa*, který napodobuje jeho vlastnosti i chování. Slovem *těleso* se označuje hmotný předmět, který má své **místo** (či adresu), **velikost** v prostoru a také **čas** vzniku i zániku. Za *těleso* lze považovat libovolný konkrétní předmět (např. dům nebo veverku), mezi *tělesa* však nepatří abstraktní pojmy jako chuze nebo spánek. Tělesa mají své **vlastnosti** (např. hmotnost, barvu nebo teplotu) a mohou poskytovat také **služby**. Příkladem může být automat na jízdenky, který za určitých podmínek vytiskne cestovní lístek. Pojem *těleso* pokrývá podmnožinu objektů reálného světa, ale kvůli některým podobnostem s počítačovými objekty ho připomeneme za chvíli.

4.10 Programový objekt

Programový objekt (v našem případě *Java objekt*) je datový **model** uložený v operační paměti počítače, který popisuje reálné či abstraktní objekty reálného světa pomocí *dat* a *algoritmů*. Popisuje reálné či abstraktní věci pomocí *dat* a *algoritmů*. Objekt může představovat konkrétní auto nebo třeba jen zelenou barvu. Podobně jako těleso má i počítačový objekt svou **adresu** v paměti, **velikost**



Obrázek 7. Diagram tříd

Podle diagramu připravíme nové třídy a upravíme také tu původní metodu pro vytvoření objektu jednoho auta. Podívejme se na výsledek...

Zdrojový kód 10. Komplexní sestavení objektu

```

public Car createCar() {

    Car car = new Car();
    car.setMade(LocalDate.of(2010, 10, 22)); ❶

    VehicleModel model = new VehicleModel();
    model.setManufacturer(new Company("Toyota", Locale.JAPAN)); ❷
    model.setName("Yaris");
    model.setTrunkVolume(436);
    car.setModel(model); ❸

    User user = new User();
    user.setPersonalNumber(10);
    user.setFirstName("Donald");
    user.setSurname("Choresidsi");
    user.setBirthDate(LocalDate.of(1990, Month.OCTOBER, 30)); ❹
    car.setOwner(user); ❺

    return car;
}

```

- ❶ Vytvoření objektu pro datum.
- ❷ Konstruktor se dvěma parametry. Atribut `JAPAN` je statický. Objekt lze vytvořit a rovnou předat jako parametr bez ukládání do proměnné.
- ❸ Propojíme model s autem.
- ❹ Datum lze vytvořit i pomocí konstanty reprezentující měsíc.
- ❺ Propojíme uživatele s autem jako vlastníka.


```

public void printCar() {
    Car car = this.createCar(); ❶

    LocalDate made = car.getMade(); ❷
    String manufacturer = car.getModel().getManufacturer().getName(); ❸
    String modelName = car.getModel().getName(); ❹
    Integer trunkVolume = car.getTrunkVolume();
    String firstname = car.getOwner().getFirstname();
    String surname = car.getOwner().getSurname();
    String owner = firstname + " " + surname; ❺

    MyPrinter.print(made, manufacturer,
        modelName, trunkVolume, firstname, surname, owner); ❻
    return; ❼
}

```

- ❶ Zavoláme metodu pro vytvoření auta. `this` znamená, že voláme metodu na stejném objektu, kde se zrovna nacházíme. Slovo `this` lze v při volání metod zpravidla vynechat.
- ❷ Načtení data výroby do proměnné `made`.
- ❸ Ukázka řetězení metod: postupně se „prokoušeme“ až k názvu výrobce. `car.getModel()` nám dá model, na něm zavoláme `getManufacturer()` a nakonec `getName()`. Chybný název metody odhalí kompilátor nebo chytrý editor.
- ❹ Další ukázka řetězení.
- ❺ Spojení tří textů do jednoho (celé jméno).
- ❻ Předání dat k tisku.
- ❼ Příkaz `return` ukončí metodu. Na konci metody s návratovým typem `void` se ten příkaz psát nemusí.

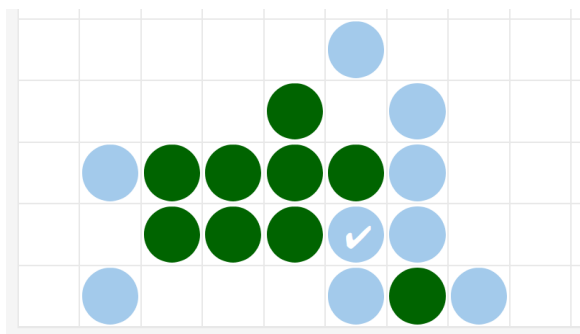
Všimněte si, že v diagramu přibyla nová třída popisující typ energie automobilu (například benzin, nafta, elektřina z baterie). Může nás také zajímat, jak se zachová volání *metody* `getName()` v případě, že metoda `getManufacturer()` **neposkytuje** žádný přiřazený objekt. Volání *metod* na neexistujícím objektu vyvolá chybu při běhu programu, která předčasně ukončí provádění *metody*. Jak už jsme zmínili dříve, hodnota prázdných objektů se v kódu popisuje klíčovým slovem `null`, a takové prázdné objekty jsou výchozí (anglicky *default*) hodnotou každého *objektového atributu* či *pole*. Přesněji bychom mohli říci, že tento případ vede k vyhození *výjimky*. Více informací o *výjimkách* (anglicky *exceptions*) se dozvíme v kapitole [Výjimky](#). Hodnotu `null` lze také zapisovat do proměnných, posílat do *metody*, či použít při porovnání instancí. Příkládám několik příkladů použití.

- 1 Nejprve si připravíme číslo, jehož bitová hodnota bude o jedničku větší než maximální hodnota čísla čtyřbitového rozsahu.
- 2 Operátor `|` (česky *svislík*, *kolmítka*, *roura*, anglicky *pipe*) provádí *bitový součet* dvou čísel, což znamená, že do výsledku promítne *logický součet bitů* na stejné pozici. Je pravda, že náš výsledek bude o jeden znak delší, ale správnou hodnotu získáme snadno metodou `substring()`. Pro úplnost uvedme také doplňkový operátor `&` pro *bitový součin*, který provádí *logický součin bitů*.

Závěrem provedme ve zdrojovém kódu vlastní změny: rozsah znakové sady rozšířme o **jeden bit** tak, aby sada pojala dvojnásobný počet znaků. Dále připravme model na vkládání znakové sady **konstruktorem**, přičemž do konstrukturu vložíme kontrolu velikosti znakové sady, která bude vyhazovat výjimku typu `IllegalArgumentException`.

7.5.4 Piškvorky s defenzivní strategií hry

Hra Piškvorky patří mezi **strategické hry**, jejichž kořeny sahají hluboko do historie. Existuje několik variant, mezi nejznámější patří zřejmě hra *Gomoku* s alternativním názvem *5 in row* (v překladu 5 v řadě). Naše pravidla hry budou následující: hráči kladou střídavě barevné kameny do mřížky s konečnou velikostí, každý hráč má svoji barvu. Vítězem se stává hráč, který jako první postaví nepřerušenou řadu alespoň pěti kamenů své barvy, povolena je i řada šikmá. Hru je možné kdykoli resetovat tlačítkem a zahájit hru novou. V naší implementaci bude protihráčem vždy počítač a výhodu prvního tahu dostane vždy člověk.

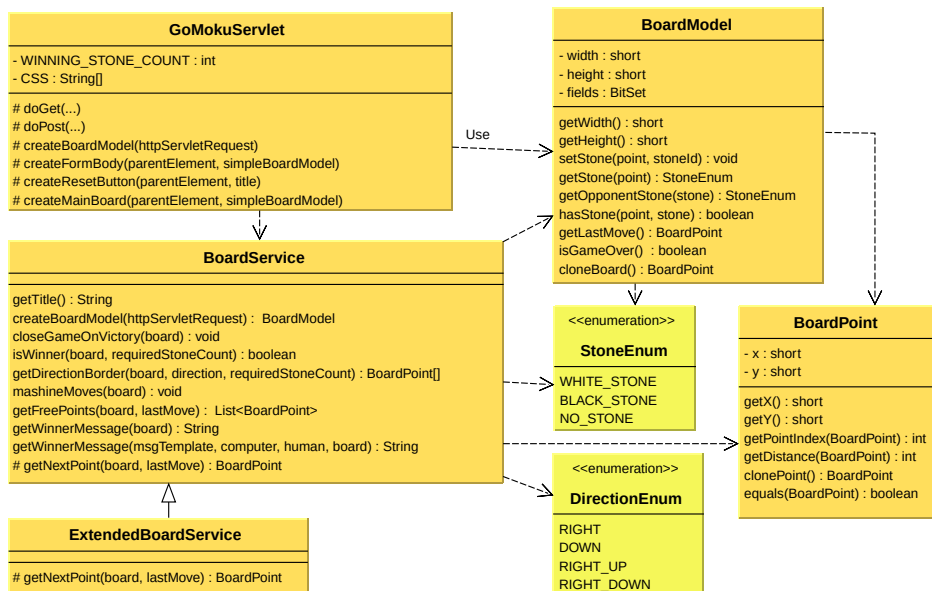


Obrázek 36. Náhled hrací desky

Strategie SW protihráče bude pouze jednoduchá obrana. Cílem hry totiž není vytvořit neporazitelný algoritmus, ale především ukázat řešení vhodné pro další studium a později třeba i pro implementaci vlastních nápadů. Čtenáři mohou nad rámec této knihy vyhledat některou knihovnu zaměřenou na hry typu *Gomoku* a zkusit napojit její algoritmy do stávající architektury.

Diagram tříd

Začneme diagramem tříd, který znázorňuje jen vybrané atributy a veřejné metody:



Obrázek 37. Diagram tříd hry Piškvorky

Na první pohled si můžeme všimnout nového typu objektu, který má nad svým názvem uveden *stereotyp* zvaný *enumeration* a místo atributů má *statické konstanty* (poznáme je podle velkých písmen). Nejdříve se podívejme na schematický popis zpracování HTTP požadavku, budeme pokračovat popisem enumerátorů a jejich možností a na konci kapitoly najdeme podrobnější popis tříd a jejich metod.

Jak to funguje?

Popis zpracování HTTP požadavku krok po kroku:

1. Servlet dostane **HTTP** požadavek metodou **GET** nebo **POST** a zavolá službu, která provede následující akce:
 - a. obnoví původní *datový model* deskové hry,
 - b. přidá do něj poslední tah hráče,
 - c. pokud tah vede k vítězství, stav se zapíše do modelu a metoda končí,
 - d. vypočítá tah počítače a přidá ho do modelu,